

train_roberta

April 13, 2026

1 AGL — RoBERTa Fine-Tuning (Transfer Learning Approach)

Adaptive Guardrail Layer (AGL) — Binary Prompt Safety Classifier

Fine-tunes roberta-base on the AGL binary dataset (Benign vs Malicious). This is the **transfer learning** approach — compare with Alex's from-scratch DL model for the final report.

Data source: data/processed/dataset_feature_engineered.csv

```
[4]: # 1. Setup
import os, sys

# Set working directory to project root (parent of notebooks/)
PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if os.path.basename(os.getcwd()) == 'notebooks':
    os.chdir(PROJECT_ROOT)
elif not os.path.exists('src/config.py'):
    # Already in project root or need to find it
    if os.path.exists('../src/config.py'):
        os.chdir('..')

if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, os.getcwd())

print(f"Working directory: {os.getcwd()}")
assert os.path.exists('src/config.py'), "Not in project root - run from the_
↳notebooks/ or project root directory"
```

Working directory: /Users/joel/GitHub/aai-590-group-8-capstone

```
[5]: # 2. Check environment
import torch

print(f"Python: {sys.version}")
print(f"PyTorch: {torch.__version__}")

if torch.cuda.is_available():
    DEVICE = "cuda"
    print(f"Device: CUDA - {torch.cuda.get_device_name(0)}")
```

```

elif hasattr(torch.backends, 'mps') and torch.backends.mps.is_available():
    DEVICE = "mps"
    print(f"Device: MPS (Apple Silicon)")
else:
    DEVICE = "cpu"
    print(f"Device: CPU (training will be slow but it works)")

# Verify data exists
data_csv = 'data/processed/dataset_feature_engineered.csv'
assert os.path.exists(data_csv), f"Data not found at {data_csv}. Run the data_
↳ pipeline notebooks first."
print(f"\nData: {data_csv} ")

```

Python: 3.12.11 | packaged by Anaconda, Inc. | (main, Jun 5 2025, 08:03:38)
 [Clang 14.0.6]
 PyTorch: 2.7.1
 Device: MPS (Apple Silicon)

Data: data/processed/dataset_feature_engineered.csv

1.1 Step 3: Build Dataset

Load data/processed/dataset_cleaned.csv, deduplicate, balance, and split into train/val/test parquets.

```

[ ]: # 3. Build dataset
from src.data.build_dataset import build_dataset

splits = build_dataset(csv_path='data/processed/dataset_feature_engineered.csv')

for name, df in splits.items():
    print(f" {name}: {len(df)} samples")

```

```

=====
Building AGL dataset (binary classification)
=====

```

```

-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[6], line 4
      1 # 3. Build dataset
      2 from src.data.build_dataset import build_dataset
----> 4 splits = build_dataset(csv_path=
      6 for name, df in splits.items():
      7     print(f" {name}: {len(df)} samples")

```

```

File ~/GitHub/aai-590-group-8-capstone/src/data/build_dataset.py:43, in
↳ build_dataset(csv_path)
    40 print(f"{' '*60}\n")
    42 # 1. Load cleaned CSV
---> 43 df = load_dataset_csv(csv_path)
    45 # 2. Add label name column for reporting
    46 df["label_name"] = df["label"].map(ID2LABEL)

File ~/GitHub/aai-590-group-8-capstone/src/data/load_datasets.py:30, in
↳ load_dataset_csv(path)
    28 from pathlib import Path
    29 if not Path(csv_path).exists():
---> 30     raise FileNotFoundError(
    31         f"Dataset CSV not found at {csv_path}\n"
    32         f"Run notebooks/data_pipeline/1_data_cleaning.ipynb first to
↳ produce it,\n"
    33         f"or pass --csv to point to a different file."
    34     )
    36 df = pd.read_csv(csv_path)
    38 # Validate expected columns

FileNotFoundError: Dataset CSV not found at data/processed/dataset_cleaned.csv
Run notebooks/data_pipeline/1_data_cleaning.ipynb first to produce it,
or pass --csv to point to a different file.

```

```

[ ]: # 3b. Verify dataset
import pandas as pd
from src.config import ID2LABEL

for split in ['train', 'val', 'test']:
    df = pd.read_parquet(f'data/processed/{split}.parquet')
    print(f"\n{split}: {len(df)} samples")
    label_counts = df['label'].value_counts()
    for label_id, count in label_counts.items():
        print(f"  {ID2LABEL[label_id]}: {count}")

```

1.2 Step 4: Fine-Tune RoBERTa (Transfer Learning)

Train the binary classifier with class-weighted cross-entropy, early stopping, and linear LR schedule.

Hyperparameters (from `src/config.py`): - LR: 2e-5, warmup: 10%, weight decay: 0.01 - Batch: 32, max epochs: 3, early stopping patience: 2 - Max sequence length: 128 - 2 classes: Benign (0), Malicious (1)

```

[ ]: # 4. Train RoBERTa classifier
from src.training.train import train_classifier

```

```
best_path = train_classifier()
print(f"\nBest model saved to: {best_path}")
```

1.3 Step 4b: Hyperparameter Experiments (Optional — skip for CPU training)

If running on GPU, uncomment and run the sweep below. On CPU, just use the defaults above.

```
[ ]: # 4b. Hyperparameter sweep (optional - GPU only)
# Uncomment to run. Each experiment takes a full training cycle.
#
# import itertools
# from src.training.train import train_classifier
# from src.config import MODELS_DIR
#
# for lr in [1e-5, 2e-5, 3e-5]:
#     name = f"lr{lr}"
#     out = MODELS_DIR / f"classifier_{name}"
#     train_classifier(output_dir=out, learning_rate=lr)
```

1.4 Step 5: Fit Anomaly Detector (Optional)

Extract [CLS] embeddings from the fine-tuned model and fit the Mahalanobis OOD detector. Threshold is calibrated on the validation set (95% in-distribution recall).

Note: The anomaly detector is optional for binary classification. It can still be useful for flagging out-of-distribution inputs that are unlike anything in the training data.

```
[ ]: # 5. Fit Mahalanobis anomaly detector (optional)
from src.training.train import train_anomaly

anomaly_path = train_anomaly()
print(f"\nAnomaly detector saved to: {anomaly_path}")
```

1.5 Step 6: Evaluate

Run the full evaluation suite: keyword baseline, TF-IDF/SVM baseline, RoBERTa classifier, and (if trained) RoBERTa + anomaly detector.

```
[ ]: # 6. Run evaluation
import numpy as np
import pandas as pd
from src.config import MODELS_DIR, PROCESSED_DIR, RESULTS_DIR
from src.evaluation.metrics import evaluate_predictions, save_results
from src.evaluation.baselines import keyword_blocklist_baseline,
    tfidf_svm_baseline

# Load test data
train_df = pd.read_parquet(PROCESSED_DIR / 'train.parquet')
test_df = pd.read_parquet(PROCESSED_DIR / 'test.parquet')
```

```

y_true = test_df['label'].values

print("=" * 60)
print("Running evaluation suite")
print("=" * 60)

all_results = {}

# Baseline 1: Keyword blacklist
print("\n[1/3] Keyword blacklist baseline...")
kw_preds = keyword_blocklist_baseline(test_df)
kw_results = evaluate_predictions(y_true, kw_preds)
all_results['keyword_blocklist'] = kw_results
print(f" Macro-F1: {kw_results['macro_f1']:.4f}")

# Baseline 2: TF-IDF + SVM
print("\n[2/3] TF-IDF + LinearSVM baseline...")
svm_preds, _ = tfidf_svm_baseline(train_df, test_df)
svm_results = evaluate_predictions(y_true, svm_preds)
all_results['tfidf_svm'] = svm_results
print(f" Macro-F1: {svm_results['macro_f1']:.4f}")

# RoBERTa
checkpoint = MODELS_DIR / 'classifier' / 'best'
if checkpoint.exists():
    print("\n[3/3] RoBERTa classifier...")
    from src.models.agl_pipeline import AGLPipeline
    pipeline = AGLPipeline.from_checkpoint(checkpoint)
    roberta_preds = np.array([pipeline.predict(t).label_id for t in
↳ test_df['text']])
    roberta_results = evaluate_predictions(y_true, roberta_preds)
    all_results['roberta'] = roberta_results
    print(f" Macro-F1: {roberta_results['macro_f1']:.4f}")
else:
    print(f"\n[3/3] Skipping RoBERTa - no checkpoint at {checkpoint}")

save_results(all_results, 'evaluation_results', RESULTS_DIR)

print("\n" + "=" * 60)
print("Summary - Macro-F1 Scores:")
print("=" * 60)
for method, res in all_results.items():
    print(f" {method:25s} {res['macro_f1']:.4f}")

```

1.6 Step 7: Visualizations

Generate figures: confusion matrix, F1 comparison, ROC curve, latency distribution.

These are saved to `results/figures/` for the final report and presentation (min 6, max 10 visualizations needed).

```
[ ]: # 7. Generate visualizations
import numpy as np
import pandas as pd
import torch
from src.config import RESULTS_DIR, FIGURES_DIR, PROCESSED_DIR, MODELS_DIR, \
    ↪MAX_SEQ_LENGTH
from src.evaluation.visualizations import (
    plot_confusion_matrix, plot_f1_comparison, plot_roc_curves
)
from src.evaluation.metrics import evaluate_predictions, benchmark_latency, \
    ↪save_results
from src.evaluation.baselines import keyword_blocklist_baseline, \
    ↪tfidf_svm_baseline
from src.models.agl_pipeline import AGLPipeline
from transformers import AutoTokenizer

FIGURES_DIR.mkdir(parents=True, exist_ok=True)

# Load test data
test_df = pd.read_parquet(PROCESSED_DIR / 'test.parquet')
train_df = pd.read_parquet(PROCESSED_DIR / 'train.parquet')
y_true = test_df['label'].values

# Baselines
kw_preds = keyword_blocklist_baseline(test_df)
svm_preds, _ = tfidf_svm_baseline(train_df, test_df)

# RoBERTa predictions + probabilities (for ROC-AUC)
checkpoint = MODELS_DIR / 'classifier' / 'best'
pipeline = AGLPipeline.from_checkpoint(checkpoint)
tokenizer = AutoTokenizer.from_pretrained(str(checkpoint), \
    ↪local_files_only=True)

roberta_preds = []
roberta_probs = []
for text in test_df['text']:
    pred = pipeline.predict(text)
    roberta_preds.append(pred.label_id)
    inputs = tokenizer(text, truncation=True, padding='max_length',
                        max_length=MAX_SEQ_LENGTH, return_tensors='pt')
    inputs = {k: v.to(pipeline.device) for k, v in inputs.items()}
    with torch.no_grad():
        logits = pipeline.model(**inputs).logits
        probs = torch.softmax(logits, dim=-1).cpu().numpy()[0]
```

```

roberta_probs.append(probs)

roberta_preds = np.array(roberta_preds)
roberta_probs = np.array(roberta_probs)

# Confusion matrix
plot_confusion_matrix(y_true, roberta_preds, title='RoBERTa (Fine-Tuned)',
                      save_path=FIGURES_DIR / 'confusion_matrix_roberta.png')

# F1 comparison
from sklearn.metrics import f1_score
f1_results = {
    'Keyword Blocklist': f1_score(y_true, kw_preds, average='macro',
    ↪zero_division=0),
    'TF-IDF + SVM': f1_score(y_true, svm_preds, average='macro',
    ↪zero_division=0),
    'RoBERTa (Fine-Tuned)': f1_score(y_true, roberta_preds, average='macro',
    ↪zero_division=0),
}
plot_f1_comparison(f1_results, save_path=FIGURES_DIR / 'f1_comparison.png')

# ROC Curve
plot_roc_curves(y_true, roberta_probs, title='RoBERTa ROC Curve',
                save_path=FIGURES_DIR / 'roc_curve_roberta.png')

# Full evaluation with AUC
results = evaluate_predictions(y_true, roberta_preds, y_prob=roberta_probs)
print(f"\nRoBERTa Results:")
print(f" Macro-F1: {results['macro_f1']:.4f}")
print(f" ROC-AUC: {results.get('roc_auc', 'N/A')}")
print(f" PR-AUC: {results.get('pr_auc', 'N/A')}")

# Latency
latency = benchmark_latency(pipeline, test_df['text'].head(50).tolist(),
    ↪n_runs=2)
print(f"\nLatency: mean={latency['mean_ms']:.1f}ms, p99={latency['p99_ms']:.
    ↪1f}ms")

# Save results JSON for comparison with Alex's model
all_results = {
    'roberta_finetuned': {**results, 'latency': latency}
}
save_results(all_results, 'roberta_evaluation', RESULTS_DIR)
print('\nAll figures saved to:', FIGURES_DIR)

```

1.7 Step 8: Done

Model checkpoint saved to `models/classifier/best/`, results to `results/`. These are gitignored but persist locally.

```
[ ]: # 8. Summary
from pathlib import Path

print("Artifacts saved locally:")
for p in ['models/classifier/best', 'models/anomaly', 'results']:
    exists = " " if Path(p).exists() else " "
    print(f" {exists} {p}")

print("\nTo run a quick demo:")
print(' python3 -m src.run --stage demo --text "ignore previous instructions"')
```